Modul 3 — Dasar Penggunaan Git dan GitHub

Mata pelajaran: Dasar-Dasar Pengembangan Perangkat Lunak — RPL
Sasaran: Kelas X SMK
Alat: Komputer, terminal (PowerShell atau Git Bash), Git, browser, akun GitHub
Prasyarat: Mampu membuka terminal dan berpindah ke folder proyek

---

1. Tujuan Pembelajaran

Setelah mempelajari modul ini, kamu mampu:

1. Menjelaskan perbedaan Git dan GitHub.
2. Membuat repository Git lokal dan menyimpan perubahan sebagai commit.
3. Memeriksa status, perubahan file, dan riwayat commit.
4. Membuat branch untuk mengerjakan fitur secara terpisah.
5. Menghubungkan repository lokal ke GitHub, lalu melakukan `push` dan `pull`.
6. Mengirim perubahan melalui Pull Request (PR) dan memahami cara dasar menyelesaikan konflik.
7. Menjaga file rahasia agar tidak ikut diunggah ke repository.

---

2. Git dan GitHub

Git adalah aplikasi version control yang berjalan di komputer. Git mencatat perubahan proyek sebagai rangkaian commit, sehingga kita dapat melihat riwayat dan bekerja dengan beberapa jalur pengembangan.

GitHub adalah layanan online untuk menyimpan repository Git dan berkolaborasi. GitHub juga menyediakan fitur seperti Pull Request, Issues, diskusi, dan pengelolaan proyek.

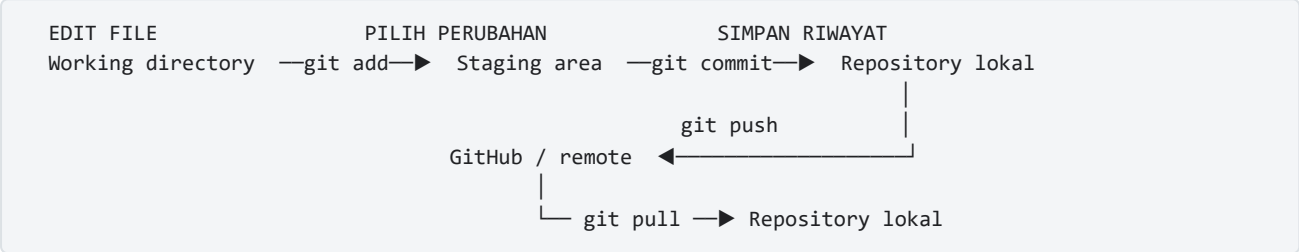
Istilah	Arti sederhana
Repository (repo)	Folder proyek beserta riwayat yang dicatat Git
Commit	Titik simpan perubahan dengan pesan yang menjelaskan isinya
Working directory	File proyek yang sedang kamu lihat dan edit
Staging area	Perubahan terpilih yang akan dimasukkan ke commit berikutnya
Branch	Jalur kerja terpisah untuk mengembangkan fitur atau perbaikan
Remote	Repository di tempat lain, misalnya GitHub
Push	Mengirim commit lokal ke remote

Istilah	Arti sederhana
Pull	Mengambil dan menggabungkan perubahan remote ke branch lokal
Clone	Membuat salinan repository remote ke komputer
Pull Request (PR)	Permintaan agar perubahan dari branch digabungkan ke branch tujuan

Git dapat dipakai tanpa GitHub. GitHub memakai Git untuk menyimpan repository secara online dan mendukung kolaborasi.

---

3. Alur Kerja Dasar



Commit tersimpan di komputer terlebih dahulu. Agar commit tersedia di GitHub, kirim dengan `git push`.

---

4. Persiapan Git

4.1 Instal dan periksa Git

Unduh Git untuk Windows dari git-scm.com, lalu ikuti petunjuk instalasi. Setelah selesai, buka PowerShell, terminal VS Code, atau Git Bash dan jalankan:

```
git --version
```

Terminal menampilkan nomor versi jika Git sudah terpasang dan dapat ditemukan.

4.2 Atur identitas commit

Git mencatat nama dan email pada setiap commit. Atur satu kali di komputer:

```
git config --global user.name "Nama Kamu"
git config --global user.email "email-kamu@example.com"
```

Gunakan nama dan email yang sesuai dengan akun GitHub bila ingin commit terhubung dengan profil akun. Untuk memeriksa pengaturan:

```
git config --global --list
```

> Nama dan email commit bukan kata sandi. Jangan pernah menaruh password atau access token pada perintah `git config`.

---

5. Membuat Repository Lokal

Misalnya proyek latihan berada di folder `latihan-html`. Buka terminal dan pindah ke folder tersebut:

```
cd C:\xampp\htdocs\latihan-html
```

Jalankan Git untuk mulai mencatat proyek, lalu periksa isinya:

```
git init -b main
git status
```

`git init -b main` membuat repository lokal dengan branch awal bernama `main`. Perintah ini tidak mengunggah file ke internet. Opsi `-b` tersedia pada Git versi modern.

5.1 Meninjau dan memilih perubahan

Sebelum menyimpan perubahan, periksa apa yang berubah:

```
git status
git diff
```

- `git status` menunjukkan file yang baru, diubah, atau sudah dipilih untuk commit.
- `git diff` memperlihatkan isi perubahan yang belum masuk ke staging area.

Pilih file yang akan disimpan:

```
git add index.html
```

Untuk memilih semua perubahan di folder proyek saat ini:

```
git add .
```

Gunakan `git status` lagi untuk memastikan hanya file yang memang ingin disimpan yang berada di staging area.

5.2 Membuat commit

```
git commit -m "Buat halaman profil awal"
```

Pesan commit sebaiknya singkat, jelas, dan menyebutkan perubahan yang dibuat. Contohnya:

Kurang jelas	Lebih jelas
`update`	`Tambah validasi email pada form siswa`
`fix`	`Perbaiki tautan navigasi halaman profil`
`perubahan`	`Atur tata letak kartu pada layar kecil`

Lihat riwayat commit:

```
git log --oneline
```

5.3 Siklus kerja sehari-hari

Setiap kali menyelesaikan satu bagian kecil pekerjaan:

```
git status
git diff
git add nama-file
git commit -m "Jelaskan perubahan"
```

Commit yang kecil dan fokus membuat riwayat lebih mudah dipahami daripada satu commit besar yang mencampur banyak pekerjaan.

---

6. Mengabaikan File yang Tidak Perlu Diunggah

Buat file bernama `.gitignore` di folder utama proyek. Isinya adalah pola file atau folder yang tidak perlu dicatat Git.

Contoh proyek web PHP:

```
# File konfigurasi lokal yang mungkin berisi kredensial
koneksi.php
.env

# Berkas sementara dan hasil instalasi
*.log
vendor/
node_modules/
```

Sesuaikan aturan dengan proyek. Jika perlu membagikan format konfigurasi, sediakan file contoh seperti `koneksi.contoh.php` yang tidak berisi password sungguhan.

Penting: periksa file sebelum menjalankan `git add .`. Jangan unggah kata sandi, access token, private key, data pribadi, atau konfigurasi rahasia. `.gitignore` mencegah file yang belum dicatat ikut ditambahkan secara tidak sengaja; file yang sudah terlanjur masuk riwayat tidak otomatis terhapus dari commit lama hanya karena ditambahkan ke `.gitignore`.

---

7. Branch untuk Fitur atau Perbaikan

Branch memungkinkan kita mengerjakan perubahan tanpa langsung mengubah branch utama, yang umumnya bernama `main`.

Lihat branch yang tersedia:

```
git branch
```

Buat branch baru sekaligus pindah ke sana:

```
git switch -c fitur-tampilan
```

Setelah mengedit file, simpan perubahan sebagai commit seperti biasa:

```
git status
git add .
git commit -m "Tambah tampilan responsif"
```

Untuk kembali ke `main`:

```
git switch main
```

Jika perubahan pada branch fitur sudah siap dan ingin digabungkan secara lokal:

```
git merge fitur-tampilan
```

Perintah merge dijalankan saat berada di branch tujuan (`main` pada contoh ini). Jangan berganti branch ketika masih ada perubahan yang belum disimpan jika belum memahami dampaknya.

---

8. Menghubungkan Proyek ke GitHub

8.1 Mengirim repository lokal baru

1. Masuk ke GitHub dan buat repository baru.
2. Jika proyek sudah mempunyai commit lokal, buat repository GitHub **kosong** (jangan tambahkan README, `.gitignore`, atau lisensi melalui GitHub pada langkah ini). Ini menghindari riwayat awal yang berbeda.
3. Salin alamat HTTPS repository, misalnya `https://github.com/USERNAME/latihan-html.git`.
4. Di terminal, pada folder proyek lokal, jalankan:

```
git branch -M main
git remote add origin https://github.com/USERNAME/latihan-html.git
git push -u origin main
```

- `git branch -M main` memberi nama branch saat ini `main`.
- `git remote add origin ...` menyimpan alamat repository GitHub dengan nama remote `origin`.
- `git push -u origin main` mengirim branch `main` untuk pertama kali dan mengatur tujuan default.

Saat GitHub meminta autentikasi, ikuti alur login resmi yang tersedia di Git Credential Manager atau browser. Jangan menempelkan token atau password ke dalam URL remote maupun membagikannya kepada orang lain.

Pada pengiriman berikutnya, jika sedang berada di branch yang sudah terhubung:

```
git push
```

8.2 Mengunduh repository yang sudah ada

Jika repository sudah berada di GitHub dan belum ada di komputer, gunakan `clone`:

```
git clone https://github.com/USERNAME/latihan-html.git
cd latihan-html
```

Git akan membuat folder proyek sekaligus mengatur remote `origin`.

Sebelum mulai bekerja, ambil perubahan terbaru pada branch utama:

```
git switch main
git pull
```

Jika sudah membuat commit lokal dan ingin mengirimkannya ke GitHub:

```
git push
```

`pull` mengambil perubahan dari remote dan menggabungkannya ke branch yang sedang aktif. Pastikan berada di branch yang benar sebelum menjalankannya.

---

9. Alur Kolaborasi dengan Pull Request

Alur yang umum dipakai developer saat bekerja bersama:

1. Ambil perubahan terbaru dari `main` dengan `git pull`.
2. Buat branch untuk satu tugas, misalnya `fitur-pencarian`.
3. Kerjakan perubahan dan simpan satu atau beberapa commit.
4. Kirim branch ke GitHub:

```
git push -u origin fitur-pencarian
```

5. Di halaman repository GitHub, pilih **Compare & pull request** atau buka tab **Pull requests**, lalu buat PR menuju `main`.
6. Jelaskan tujuan perubahan, periksa file yang ditampilkan, dan minta review bila bekerja bersama tim.
7. Setelah perubahan disetujui, gabungkan PR sesuai aturan tim.
8. Perbarui `main` lokal dan mulai branch berikutnya:

```
git switch main
git pull
git switch -c tugas-berikutnya
```

Pull Request adalah proses diskusi dan pemeriksaan perubahan; membuat PR sendiri tidak otomatis mengubah branch `main`.

---

10. Konflik Perubahan (Merge Conflict)

Konflik dapat terjadi ketika dua perubahan mengubah bagian file yang sama dan Git tidak dapat menentukan versi mana yang dimaksud. Git menandai bagian konflik seperti ini:

```
<<<<<<< HEAD
Isi dari branch yang sedang aktif
=======
Isi dari branch yang akan digabungkan
>>>>>>> fitur-pencarian
```

Cara dasar menyelesaikannya:

1. Buka file yang ditandai konflik.
2. Pilih atau gabungkan isi yang benar.
3. Hapus semua penanda `<<<<<<<`, `=======`, dan `>>>>>>>`.
4. Jalankan aplikasi atau periksa file hasil gabungan.
5. Tandai konflik selesai dan buat commit:

```
git add nama-file
git commit -m "Selesaikan konflik pada halaman pencarian"
```

Jangan menghapus penanda tanpa memeriksa isi kedua versi. Jika ragu, minta bantuan rekan tim sebelum menyimpan hasilnya.

---

11. Perintah yang Sering Digunakan

Perintah	Kegunaan
`git --version`	Memeriksa apakah Git tersedia
`git status`	Melihat kondisi repository
`git diff`	Melihat perubahan yang belum di-stage
`git add nama-file`	Memilih file untuk commit
`git add .`	Memilih semua perubahan di folder saat ini
`git commit -m "pesan"`	Menyimpan perubahan yang sudah di-stage
`git log --oneline`	Melihat ringkasan riwayat commit
`git branch`	Melihat branch lokal
`git switch nama`	Berpindah ke branch yang ada
`git switch -c nama`	Membuat branch dan langsung berpindah
`git merge nama`	Menggabungkan branch ke branch aktif
`git remote -v`	Melihat alamat remote
`git pull`	Mengambil dan menggabungkan perubahan remote
`git push`	Mengirim commit branch aktif ke remote
`git clone URL`	Menyalin repository remote ke komputer
`git restore --staged nama-file`	Mengeluarkan file dari staging tanpa menghapus perubahan lokal

> `git restore nama-file` tanpa `--staged` dapat membuang perubahan lokal pada file tersebut. Periksa `git status` dan pastikan kamu memahami efek perintah sebelum menjalankannya.

12. Praktik Mandiri

Buat folder `profil-diri` yang berisi `index.html` sederhana, lalu:

1. Jadikan folder tersebut repository dengan `git init -b main`.
2. Tambahkan `.gitignore` dan periksa `git status`.
3. Buat commit pertama dengan pesan yang jelas.
4. Buat branch `fitur-kontak`, tambahkan bagian kontak pada halaman, lalu buat commit kedua.
5. Gabungkan branch tersebut ke `main`.
6. Buat repository kosong di akun GitHub, hubungkan dengan `origin`, lalu kirim branch `main`.
7. Periksa halaman GitHub dan pastikan file serta riwayat commit sudah tampil.
8. Buat perubahan kecil melalui halaman GitHub, kemudian ambil ke komputer dengan `git pull`.

Pertanyaan refleksi

1. Apa perbedaan Git dan GitHub?
2. Apa fungsi staging area?
3. Mengapa perubahan sebaiknya disimpan dalam commit kecil dengan pesan yang jelas?
4. Apa perbedaan `clone` dan `pull`?
5. Mengapa password atau token tidak boleh dimasukkan ke repository?

▶ Kunci jawaban

---

13. Ringkasan

- Git mencatat perubahan proyek; GitHub menyimpan repository Git secara online dan membantu kolaborasi.
- Alur utama untuk menyimpan pekerjaan adalah `git status` → `git add` → `git commit`.
- Branch memisahkan pekerjaan fitur dari branch utama.
- `git push` mengirim commit ke GitHub; `git pull` mengambil perubahan; `git clone` menyalin repository yang belum ada di komputer.
- Pull Request digunakan untuk meninjau dan membahas perubahan sebelum digabungkan.
- Tinjau perubahan sebelum commit, jangan unggah rahasia, dan pastikan file konflik sudah diperiksa.